

Paso 7 - Repetir con for | Solucionario

Las cinco soluciones del paso 7. El `for` anidado es lo que más cuesta: en todas, el **dónde va el `println`** es la mitad del ejercicio.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso07` y `javac *.java`.

B7.1 - Los múltiplos de 3

Un `for` del 1 al 100 con dos acumuladores, y después el mismo recorrido imprimiendo, con un contador para cortar la línea cada 10.

▼ Ver la solución (B7_1_Multiplos.java)

B7_1_Multiplos.java

```
public class B7_1_Multiplos {
    public static void main(String[] args) {
        int cuantos = 0;
        int suma = 0;

        for (int i = 1; i <= 100; i++) {
            if (i % 3 == 0) {
                cuantos++;
                suma += i;
            }
        }

        System.out.println("Multiplos de 3 entre 1 y 100: " + cuantos);
        System.out.println("Suman: " + suma);
        System.out.println();

        int enLaLinea = 0;
        for (int i = 3; i <= 100; i += 3) {           // partir en 3 y avanzar de 3
en 3
            System.out.print(i + " ");
            enLaLinea++;
            if (enLaLinea == 10) {
                System.out.println();
                enLaLinea = 0;
            }
        }
        System.out.println();
    }
}
```

Salida:

Multiplos de 3 entre 1 y 100: 33

Suman: 1683

3 6 9 12 15 18 21 24 27 30

33 36 39 42 45 48 51 54 57 60

63 66 69 72 75 78 81 84 87 90

93 96 99

Dos formas de recorrer, las dos correctas. El primer `for` va de 1 en 1 y pregunta `i % 3 == 0`; el segundo parte en 3 y avanza `i += 3`, así que **todos** los números que visita sirven y no necesita `if`. El segundo da 33 vueltas en vez de 100.

El truco de las 10 por línea es un contador aparte que se reinicia. No sirve `i % 10`, porque lo que hay que contar no es el número sino **cuántos llevas impresos**.

El `println()` del final es por si la última línea quedó a medias: sin él, el símbolo del sistema aparece pegado a los números.

B7.2 - El factorial

Un acumulador que **multiplica** en vez de sumar, y por eso parte en 1 y no en 0. El programa calcula el mismo factorial en `int` y en `long` para que se vea el desborde.

▼ Ver la solución (B7_2_Factorial.java)

B7_2_Factorial.java

```
import java.util.Scanner;

public class B7_2_Factorial {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Numero: ");
        int n = sc.nextInt();

        int factorialInt = 1;
        long factorialLong = 1;

        for (int i = 1; i <= n; i++) {
            factorialInt *= i;
            factorialLong *= i;
        }

        System.out.println(n + "! en int: " + factorialInt);
        System.out.println(n + "! en long: " + factorialLong);
        System.out.println("El int llega hasta " + Integer.MAX_VALUE);

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Numero: 5
5! en int: 120
5! en long: 120
El int llega hasta 2147483647

--- ejecucion 2
Numero: 13
13! en int: 1932053504
13! en long: 6227020800
El int llega hasta 2147483647

--- ejecucion 3
Numero: 20
20! en int: -2102132736
20! en long: 2432902008176640000
El int llega hasta 2147483647

--- ejecucion 4
Numero: 21
21! en int: -1195114496
21! en long: -4249290049419214848
El int llega hasta 2147483647
```

Lo que importa: el acumulador de multiplicar parte en 1. Si lo inicias en 0, todo el factorial da 0 y el error es invisible (no hay ningún mensaje).

El desborde. Desde el 13 el `int` entrega un número **inventado** y nadie avisa: 6227020800 no cabe en 32 bits, así que Java se queda con los bits de más abajo. En el 20 el `int` ya da negativo, y en el **21** hasta el `long` se desborda. No existe ningún aviso del compilador ni excepción: es un resultado malo que se ve igual de convincente que uno bueno.

Esta es la razón de fondo por la que en programas de plata nunca se usa `double` ni `int` para los pesos: se usan enteros grandes o tipos decimales exactos. Por ahora, la lección es más simple: **cuando un número crece rápido, elige `long`**.

B7.3 - Las figuras

Las tres, con el mismo esqueleto: el `for` de afuera son las filas, el de adentro lo que se dibuja en cada una.

▼ Ver la solución (B7_3_Figuras.java)

B7_3_Figuras.java

```

import java.util.Scanner;

public class B7_3_Figuras {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Tamano: ");
        int n = sc.nextInt();

        System.out.println("\nCreciente:");
        for (int fila = 1; fila <= n; fila++) {
            for (int col = 1; col <= fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nDecreciente:");
        for (int fila = n; fila >= 1; fila--) {
            for (int col = 1; col <= fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }

        System.out.println("\nPiramide:");
        for (int fila = 1; fila <= n; fila++) {
            for (int espacio = 1; espacio <= n - fila; espacio++) {
                System.out.print(" ");
            }
            for (int estrella = 1; estrella <= 2 * fila - 1; estrella++) {
                System.out.print("*");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

Salida:

Tamano: 4

Creciente:

```
*  
**  
***  
****
```

Decreciente:

```
****  
***  
**  
*
```

Piramide:

```
  *  
  **  
 ***  
****  
*****
```

Las tres fórmulas.

- Creciente: la fila `f` dibuja `f` estrellas.
- Decreciente: es la misma, pero el `for` de afuera va **al revés** (`fila--`). No hace falta cambiar nada adentro.
- Pirámide: cada fila lleva `n - fila` espacios y `2 * fila - 1` estrellas (1, 3, 5, 7: siempre impar, por eso queda centrada).

Si la pirámide te quedó torcida, imprime un `.` en vez del espacio: se ve al instante cuántos espacios está poniendo de más o de menos. Es la forma más rápida de depurar cualquier dibujo.

B7.4 - La tabla completa

Un `for` dentro de otro, y `printf` para que las columnas no se muevan.

▼ Ver la solución (B7_4_TablaCompleta.java)

B7_4_TablaCompleta.java

```
public class B7_4_TablaCompleta {  
    public static void main(String[] args) {  
        for (int fila = 1; fila <= 10; fila++) {  
            for (int columna = 1; columna <= 10; columna++) {  
                System.out.printf("%4d", fila * columna);  
            }  
            System.out.println();  
        }  
    }  
}
```

Salida:

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100

Lo que importa. El `println()` va **entre** las dos llaves de cierre: al terminar cada fila, no en cada celda. Si lo pones adentro del `for` interno, salen 100 líneas de un número cada una; si lo pones después de los dos, sale todo en una sola línea. Los dos errores son clásicos y se arreglan mirando **dónde** está el `println`.

Usé `%4d` en vez de `%3d` porque el 100 necesita 3 espacios y quedaría pegado al 90. La regla: el ancho se elige mirando **el número más grande** de la tabla.

B7.5 - Desafío: los primos

Un `for` que recorre los candidatos y, dentro, otro que busca un divisor. El `break` corta la búsqueda apenas encuentra uno.

▼ Ver la solución (B7_5_Primos.java)

B7_5_Primos.java

```
import java.util.Scanner;

public class B7_5_Primos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Hasta que numero? ");
        int hasta = sc.nextInt();

        int cuantos = 0;

        for (int n = 2; n <= hasta; n++) {
            boolean esPrimo = true;

            for (int divisor = 2; divisor < n; divisor++) {
                if (n % divisor == 0) {
                    esPrimo = false;
                    break;           // ya se sabe: no sirve seguir buscando
                }
            }

            if (esPrimo) {
                System.out.print(n + " ");
                cuantos++;
            }
        }

        System.out.println();
        System.out.println("Son " + cuantos + " primos");

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Hasta que numero? 30
2 3 5 7 11 13 17 19 23 29
Son 10 primos

--- ejecucion 2
Hasta que numero? 2
2
Son 1 primos
```

El patrón: la bandera. `boolean esPrimo = true;` se declara **dentro** del `for` de afuera, porque hay que reiniciarla para cada número. Si la declaras antes de todo, el primer número compuesto la deja en `false` para siempre y desde ahí no encuentras ningún primo más.

El `break` no cambia el resultado, cambia el tiempo: sin él, un número como 1000 sigue probando 998 divisores después de saber que no es primo.

Mejora clásica, por si quieres probarla: basta con buscar divisores hasta la **raíz cuadrada** de `n` (`divisor * divisor <= n`). Si `n` tuviera un divisor mayor que su raíz, su pareja sería menor, y ya lo habrías encontrado. Con eso, buscar los primos hasta un millón pasa de minutos a un segundo.

Volver a la [hoja del paso 7](#) o al [paso 7 de la guía](#).